

CounterStock: User Manual

Winter Semester 2026

POS Operations, Inventory Control & Dispatch
Instructor: Stephan Bökelmann

THGA Bochum
Informationstechnik und Digitalisierung

Table of Contents

1	Motivation and Operational Context	2
2	System Requirements and Installation	2
2.1	Hardware and Network Prerequisites	2
2.2	Native Debian Package Deployment	2
3	Application Setup & Connection	2
4	Cashier Operations: Menu & Live Ordering	3
4.1	Real-Time Portion Calculation	3
4.2	Submitting Customer Orders	3
4.3	Handling Out-of-Stock Rejections	4
5	Stock Monitoring and Delivery Intake	5
5.1	Recording Incoming Deliveries	5
6	Frequently Asked Questions (FAQ)	6

Notice for Terminal Operators: Keep this document accessible at the cashier terminal. It covers daily setup, live portion tracking, transaction conflict diagnosis, and delivery intake workflows.

1 Motivation and Operational Context

In standard snack bars and fast-service gastronomy, point-of-sale terminals operating independently from physical storage produce phantom orders: cashiers accept payments for items whose ingredients are depleted. This results in order cancelations, refund overhead, and lost customer confidence.

CounterStock resolves this problem at the database boundary. The desktop client queries dynamically computed portions based on physical ingredient stocks, forbidding cashiers from placing orders when even a single recipe component is exhausted.

Core Principle: Cashiers do not input abstract numbers; they operate directly against a live view of real kitchen warehouse availability.

2 System Requirements and Installation

2.1 Hardware and Network Prerequisites

- **Operating System:** Ubuntu Linux 22.04 LTS / 24.04 LTS (x86_64)
- **Display:** Minimum resolution of 1024×768
- **Network:** HTTP connectivity to the CounterStock API host (<http://localhost:8000>)

2.2 Native Debian Package Deployment

The desktop GUI is distributed as an independent, pre-compiled Debian package containing the necessary Python runtime and Tkinter bindings:

```
1 # Install native .deb archive
2 sudo dpkg -i db-frontend_0.1.0_amd64.deb
3
4 # Launch application via system path
5 db-frontend
```

3 Application Setup & Connection

Upon launching `db-frontend`, a modal handshake dialog prompts the cashier or manager to provide server coordinates.

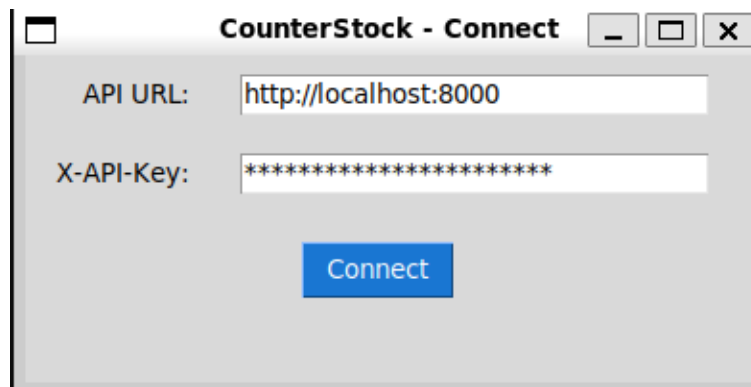


Figure 1: Modal Handshake: Server Base URL and API Token Entry

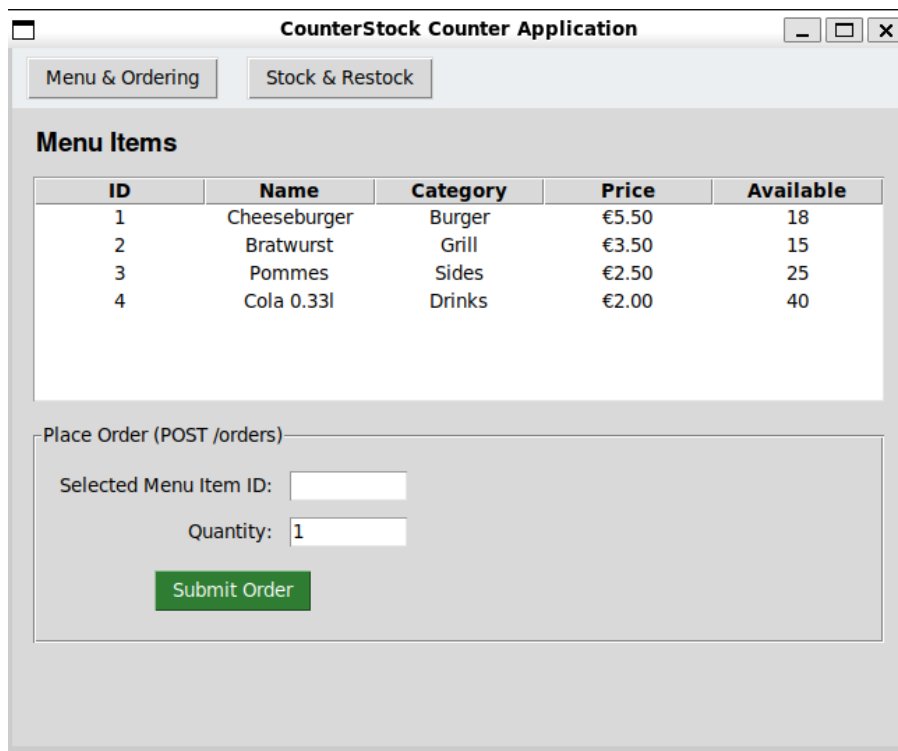
1. **API URL:** Address of the backend service (default: `http://localhost:8000`).
2. **X-API-Token:** The security token issued to the terminal (`secret_counterstock_key`).
3. Click **Connect**. When verified, the terminal transitions to active status.

Diagnostic Note: If the dialog reports a connection error, verify that the Docker Compose backend cluster is running (`docker compose ps`) and port 8000 is open.

4 Cashier Operations: Menu & Live Ordering

4.1 Real-Time Portion Calculation

Navigate to the **Menu & Ordering** tab. The table displays active menu offerings.



CounterStock Counter Application

Menu & Ordering | Stock & Restock

Menu Items

ID	Name	Category	Price	Available
1	Cheeseburger	Burger	€5.50	18
2	Bratwurst	Grill	€3.50	15
3	Pommes	Sides	€2.50	25
4	Cola 0.33l	Drinks	€2.00	40

Place Order (POST /orders)

Selected Menu Item ID:

Quantity:

Submit Order

Figure 2: Live POS Catalog Showing Dynamically Computed Available Portions

The **Available** column is not manually set. It is computed in real-time from the bottleneck ingredient:

$$AvailablePortions = \min_{r \in Ingredients} \left(\left\lfloor \frac{StockQuantity_r}{RequiredQuantity_r} \right\rfloor \right) \quad (1)$$

4.2 Submitting Customer Orders

1. Enter the **Menu Item ID** (e.g., 1 for Cheeseburger).
2. Enter the requested **Quantity**.
3. Click **Submit Order**.
4. **Success:** A green confirmation popup indicates the Order ID and calculated subtotal. The available portion count drops immediately across terminals.

4.3 Handling Out-of-Stock Rejections

If a transaction requests more items than ingredients permit, the server rejects the order:

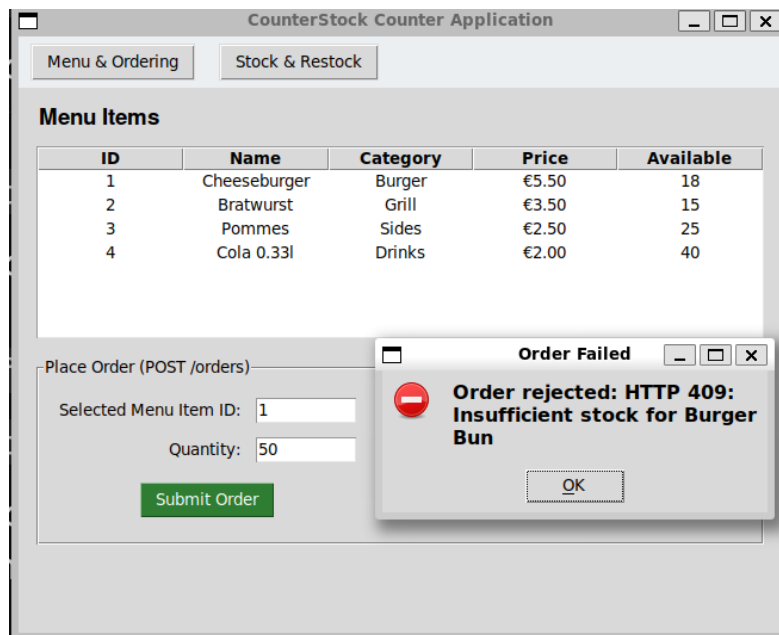


Figure 3: Out-of-Stock Conflict Rejection Notice (HTTP 409)

The cashier receives an explicit error dialog: **HTTP 409 Conflict: Insufficient stock for [Ingredient]**. Offer alternative dishes or request an ingredient restock.

5 Stock Monitoring and Delivery Intake

Switch to the **Low Stock** tab. Ingredients at or below their configured safety margin appear automatically.

ID	Name	Stock	Threshold	Unit
2	Beef Patty	9.0	10.0	pcs
4	Bratwurst Sausag	5.0	5.0	pcs
3	Cheddar Slice	4.0	5.0	pcs
6	Cola 0.33l	4.0	12.0	can
5	French Fries (Froz)	1000.0	1000.0	g

Restock Ingredient (POST /ingredients/{id}/restock)

Ingredient ID:

Amount to Add:

Figure 4: Low-Stock Threshold Alert Dashboard

5.1 Recording Incoming Deliveries

1. Select the ingredient from the Low Stock grid.
2. Enter the received delivery amount in the text field.
3. Click **Restock Inventory**. The menu catalog instantly reflects the increased available portions.

6 Frequently Asked Questions (FAQ)

Q: Why does the login screen report "Connection Refused"?

The backend API is not listening. Run `docker compose up -d` inside `counterstock-backend` and confirm health via `curl http://localhost:8000/menu`.

Q: Can inventory balances drop below zero?

No. The database enforces this via a `CHECK (stock_quantity >= 0)` constraint. Overselling is blocked by the transactional engine.

Q: What happens if the cashier window closes unexpectedly?

All confirmed sales are committed to PostgreSQL. No local state is lost; reopening `db-frontend` restores the latest live values.